Activity IV - Fundamental of Cryptography

Exercises

1. (Encryption and Statistical Analysis) Though encryption is primarily designed to preserve confidentiality and integrity of data, the mechanism itself is vulnerable to brute force (statistical analysis). In other words, the more we see the encrypted data, the easier we can hack it. In this exercise, you are asked to crack the following cipher text. Please provide the decrypted result and explain your strategy in decrypting this text.

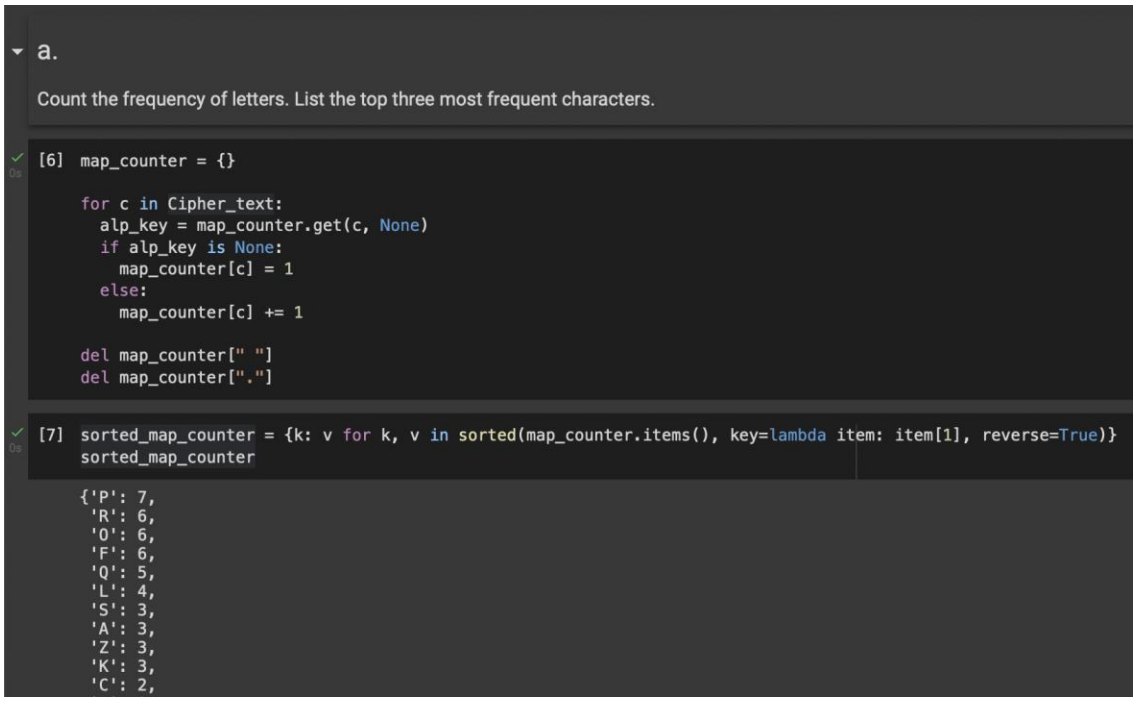
Cipher text

PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALOQSKR. QDFP FP ZK LIU BROJZK MOLTROE.

a. Count the frequency of letters. List the top three most frequent characters.

- The top three most most frequent characters are

1st	P	(7)
2nd	R, O, F	(6)
3rd	Q	(5)



b. Knowing that this is English, what are commonly used three-letter words and two-letter words. Does the knowledge give you a hint on cracking the given text?

- Commonly used
  Three-letter words: you are the and
  Two-letter words: is am in on at

- Yes, the first word I was able to decrypt is “IS” from the original “FP”.

c. Cracking the given text. Measure the time that you have taken to crack this message.

- The given text was successfully cracked. Then, we knew it was ..

“SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.”

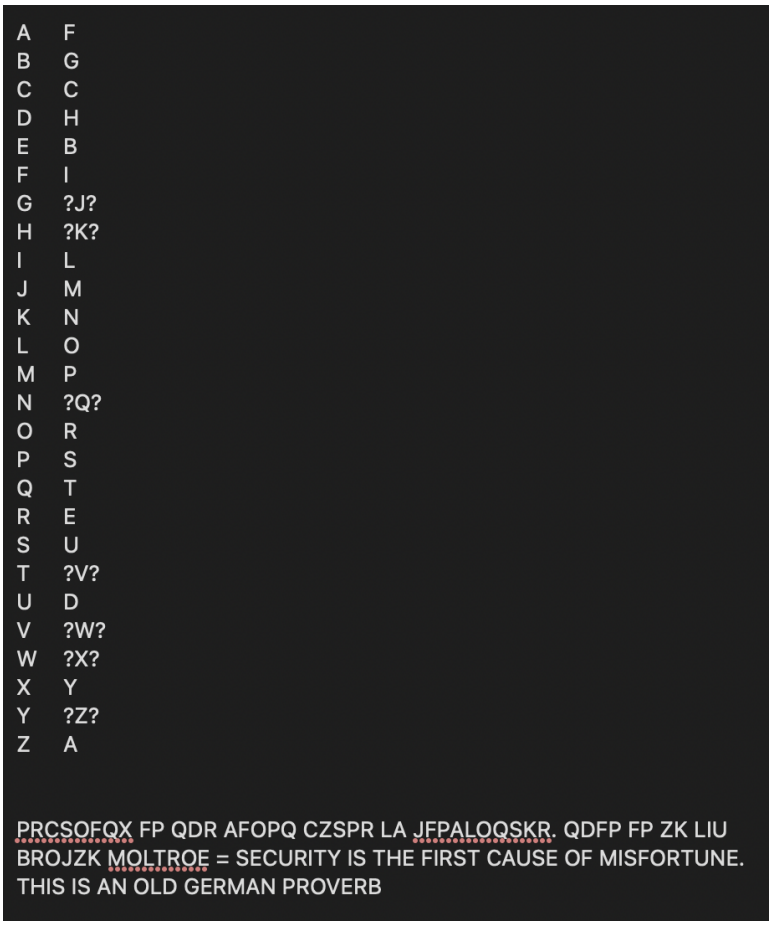
- I took about 1 hour with my colleague to manually crack these sentences.

d. Explain your process in hacking such messages.

- We started from guessing low-length words in the given text, which are “FP”, “QDR”, “QDFP”, … in sequence. Then, we took note of the mapping of encrypted alphabets to their original ones.

- After we got the original word of each encrypted word, we replaced the letter at every index of the sentences to the original alphabet of it. Then, we repeated to guess a new word from more information we had freshly accumulated.

- Finally, we got the original sentences of the messages as mentioned above.



e. If you know that the encryption scheme is based on Caesar (Monoalphabetic Substitution) that is commonly used by Caesar for sending messages to Cicero, does it allow you to crack it faster?

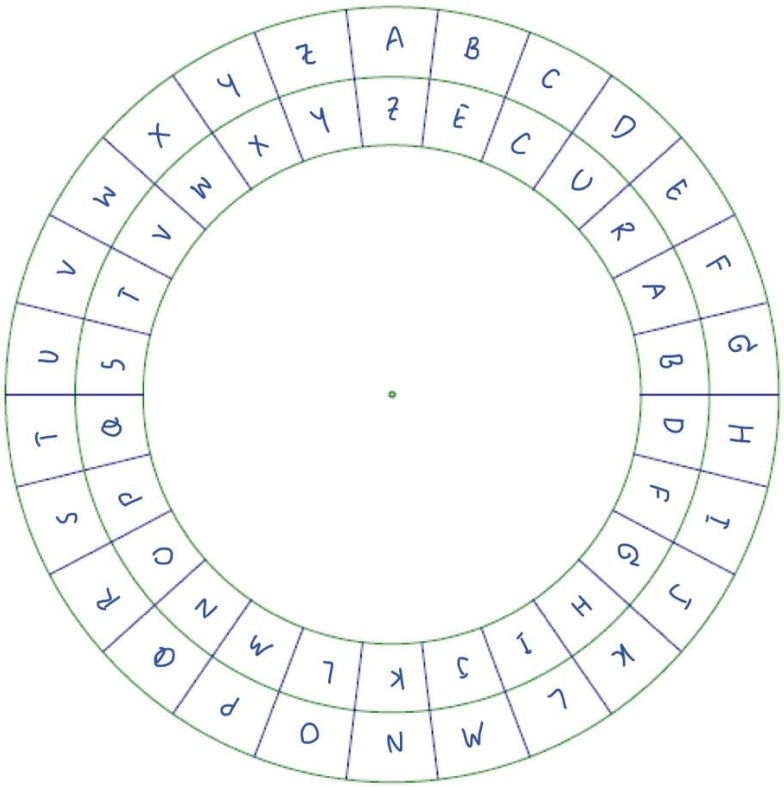
- Surely, it could help us not to guess with other types of encryption methods.

- Now, we got the mapping alphabet is ..

ABCDEFGHIJKLMNOPQRSTUVWXYZ

ZECURABDFGHIJKLMNOPQSTVWXY

f. Draw a cipher disc of the given text.



g. Create a simple python program for cracking the Caesar cipher text using brute force attack. Explain the design and demonstrate your software. (You may use an English dictionary for validating results.)

- The program will receive an input of encrypted text, then split into several encrypted words.
- Starting from low-length words, the algorithm performs for-loop from lower-length words to higher-length words matching their length to the words in a dictionary file. Then, it chooses one word from many possible words to guess the next word with the letter acquired from the selected one.
- They will repeat this process until getting the final answer, which all of the encrypted letters are decoded with the same mapping table, and the resulted words are contained in the dictionary.

2. (Symmetric Encryption) Vigenère is a complex version of the Caesar cipher. It is a polyalphabetic substitution.

   a. Based on the Caesar cipher, explain how it can be used to cipher data.

   - Based on the Caesar cipher, the Vigenère can encrypt a letter to more than one letter resulted, depending on the index of the cipher text. The encryption includes 'Key' to define the row index of mapping function for each letter, because each letter should be differently encrypted to varied alphabet by the 'Key' defined with its position.

   b. If a key is the word "CAT", please analyze the level of security provided by Vigenère compared to that of the Caesar cipher.

   - For Vigenère cipher, the key "CAT" should provide three patterns of encryption, while the Caesar could encode any message with only one pattern as we did in the exercise 1.

c. Create a python program for ciphering data using Vigenère.

- I used the same plain_text, and key from the professor's slide, and got the same cipher_text.

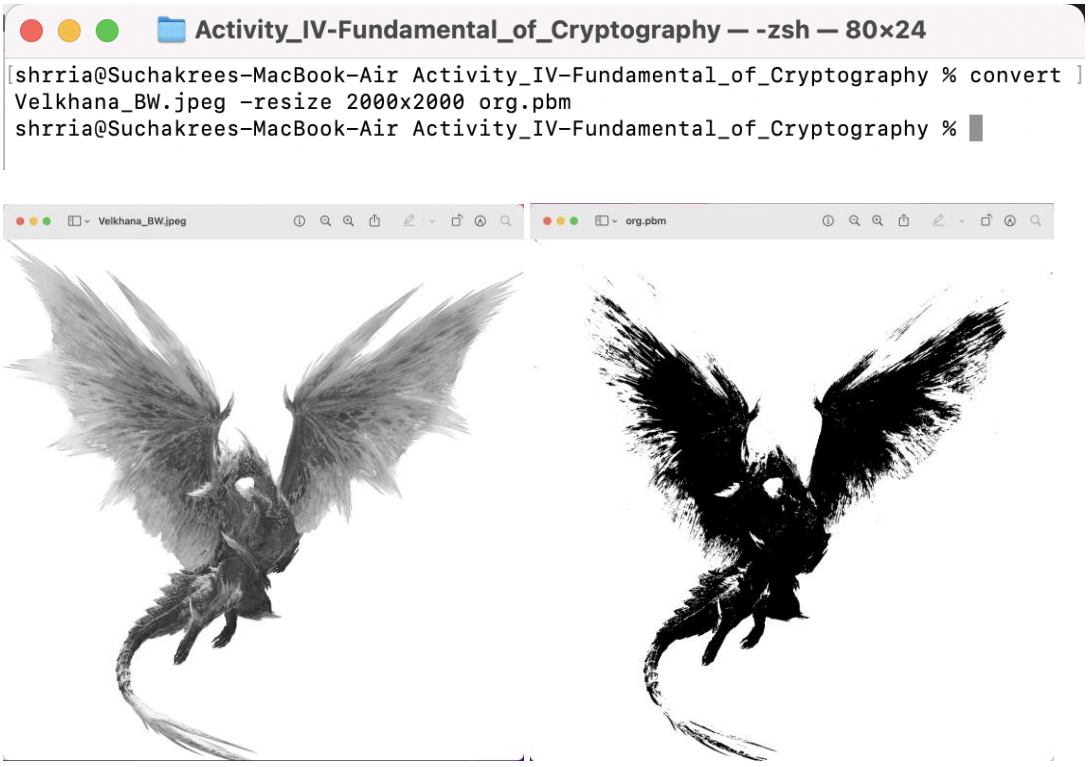
```python
import sys


def encrypt(plain_text, key):
    cipher_text = ""
    for i in range(len(plain_text)):
        x = (ord(plain_text[i]) + ord(key[i])) % 26 + ord("A")
        cipher_text += chr(x)
    return cipher_text


if __name__ == "__main__":
    plain_text = sys.argv[1]
    key = sys.argv[2]
    cipher_text = encrypt(plain_text, key)
    print("Encrypted Text:", cipher_text)

```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  JUPYTER
shrria@Suchakrees-MacBook-Air 2 % python vigenere.py COMPUTING LUCKYLUCK
Encrypted Text: NIOZSECPQ
shrria@Suchakrees-MacBook-Air 2 %
```

3. (Mode in Block Cipher) Block Cipher is designed to have more randomness in a block. However, an individual block still utilizes the same key. Thus, it is recommended to use a cipher mode with an initial vector, chaining or feedback between blocks. This exercise will show you the weakness of **E**lectronic **C**ode **B**ook mode which does not include any initial vector, chaining or feedback.

    a. Find a bitmap image that is larger than 2000x2000 pixels. Note that you may resize any image. To simplify the pattern, we will change it to bitmap (1-bit per pixel) using the portable bitmap format (pbm). In this example, we will use imagemagick for the conversion.

```
$ convert image.jpg -resize 2000x2000 org.pbm
```



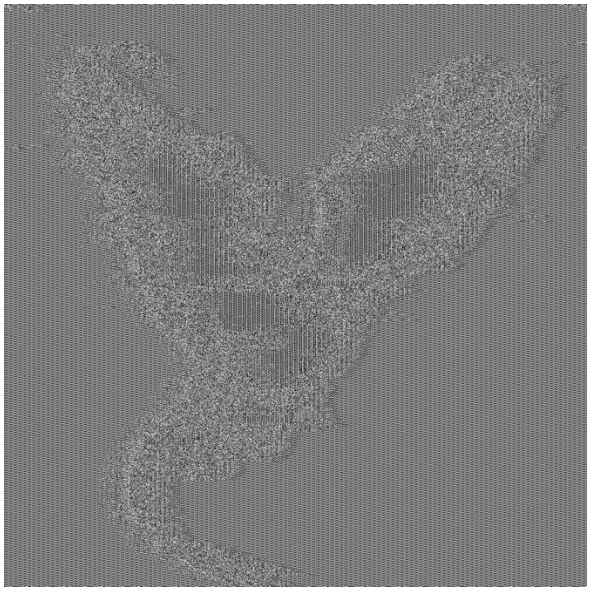
b. The NetPBM format is a naive image format.The first two lines contain a header (format and size in pixel). Depending on the format, the pixels can be represented in either binary and ascii. For our exercise, we prefer binary. However, we first have to take out the header to prevent the encryption from encoding the header. To do so, use your text editor (eg. vi, notepad) to take out the first two lines.

```
$ cp org.pbm org.x
$ vi org.x
P4
2000 2000
KR)B@HD◆◆@H◆
```

c. Encrypt the file with OpenSSL[2] with any block cipher algorithm in ECB mode (no padding and no salt).
$ openssl enc -aes-256-ecb -in org.x -nosalt -out enc.x

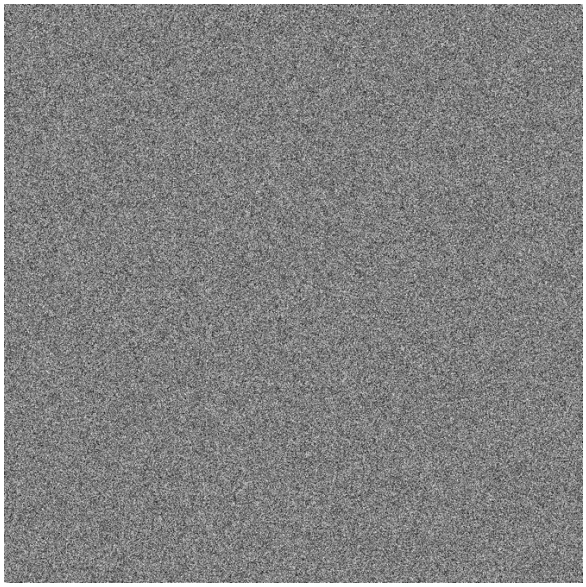
```
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % openssl enc -aes-256-ecb -in org
.x -nosalt \
-out enc.x
enter aes-256-ecb encryption password:
Verifying - enter aes-256-ecb encryption password:
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % cp enc.x enc.pbm
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % vim enc.pbm
```

d. Pad the header back and see the result.



e. You may try it with other modes with IV, chaining, or feedback and compare the result.

```
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % openssl enc -aes-256-cfb -in org
.x -nosalt \
-out enc-cfb.x
enter aes-256-cfb encryption password:
Verifying - enter aes-256-cfb encryption password:
```



f. What does the result suggest about the mode of operation in block cipher? Please provide your analysis.

- From the experiment, we've found that the encryption algorithm of ECB gave the resulted image which allowed human's eyes to see the bpm pattern was similar to the original image. Conversely, the encryption algorithm of CFB returned the image which could not be compare similarly to the original one.

4. (Encryption Protocol - Digital Signature)

a. Measure the performance of a hash function (sha1), RC4, Blowfish and DSA. Outline your experimental design.
(Please use OpenSSL for your measurement)

- SHA-1

```
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % openssl speed sha1
Doing sha1 for 3s on 16 size blocks: 20784920 sha1's in 2.99s
Doing sha1 for 3s on 64 size blocks: 12969143 sha1's in 2.99s
Doing sha1 for 3s on 256 size blocks: 5817270 sha1's in 2.99s
Doing sha1 for 3s on 1024 size blocks: 1816717 sha1's in 2.99s
Doing sha1 for 3s on 8192 size blocks: 244794 sha1's in 2.99s
LibreSSL 2.8.3
built on: date not available
options:bn(64,64) rc4(ptr,int) des(idx,cisc,16,int) aes(partial) blowfish(idx)
compiler: information not available
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes   8192 bytes
sha1           111048.69k   277342.71k   497798.22k   621677.67k   669916.37k
```

- RC4

```
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % openssl speed rc4
Doing rc4 for 3s on 16 size blocks: 197427374 rc4's in 3.00s
Doing rc4 for 3s on 64 size blocks: 59704755 rc4's in 3.00s
Doing rc4 for 3s on 256 size blocks: 15695363 rc4's in 3.00s
Doing rc4 for 3s on 1024 size blocks: 3961928 rc4's in 3.00s
Doing rc4 for 3s on 8192 size blocks: 497715 rc4's in 3.00s
LibreSSL 2.8.3
built on: date not available
options:bn(64,64) rc4(ptr,int) des(idx,cisc,16,int) aes(partial) blowfish(idx)
compiler: information not available
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes   8192 bytes
rc4            1054230.40k  1274306.74k  1340207.88k  1354347.04k  1360706.20k
```

- Blowfish

```
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % openssl speed blowfish
Doing blowfish cbc for 3s on 16 size blocks: 24815334 blowfish cbc's in 2.98s
Doing blowfish cbc for 3s on 64 size blocks: 6524743 blowfish cbc's in 2.99s
Doing blowfish cbc for 3s on 256 size blocks: 1644041 blowfish cbc's in 2.99s
Doing blowfish cbc for 3s on 1024 size blocks: 411239 blowfish cbc's in 2.99s
Doing blowfish cbc for 3s on 8192 size blocks: 51208 blowfish cbc's in 2.99s
LibreSSL 2.8.3
built on: date not available
options:bn(64,64) rc4(ptr,int) des(idx,cisc,16,int) aes(partial) blowfish(idx)
compiler: information not available
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes   8192 bytes
blowfish cbc    133023.99k   139664.21k   140607.96k   140854.92k   140348.74k
```

- DSA

```
shrria@Suchakrees-MacBook-Air Activity_IV-Fundamental_of_Cryptography % openssl speed dsa
Doing 512 bit sign dsa's for 10s: 157969 512 bit DSA signs in 9.99s
Doing 512 bit verify dsa's for 10s: 175832 512 bit DSA verify in 9.99s
Doing 1024 bit sign dsa's for 10s: 62819 1024 bit DSA signs in 9.95s
Doing 1024 bit verify dsa's for 10s: 60532 1024 bit DSA verify in 9.96s
Doing 2048 bit sign dsa's for 10s: 19994 2048 bit DSA signs in 9.97s
Doing 2048 bit verify dsa's for 10s: 18230 2048 bit DSA verify in 9.97s
LibreSSL 2.8.3
built on: date not available
options:bn(64,64) rc4(ptr,int) des(idx,cisc,16,int) aes(partial) blowfish(idx)
compiler: information not available
                  sign    verify    sign/s verify/s
dsa  512 bits 0.000063s 0.000057s  15812.5  17597.8
dsa 1024 bits 0.000158s 0.000165s   6313.1   6076.1
dsa 2048 bits 0.000498s 0.000547s   2006.3   1828.0
```

b. Comparing performance and security provided by each method.

- Performance:
From the experiments, the speed (average) of encryption algorithms can be sorted as followings:

Blowfish < DSA < SHA-1 < RC4

- Security:
From the lecture, we have known that Blowfish (Block Cipher) is stronger than RC4 (Stream Cipher). The weakest one is SHA-1 which is a hash-digest encryption, and DSA is strongest because of its asymmetricity. Thus, the algorithms of encryption can be sorted by levels of security as followings:

SHA-1 < Blowfish < RC4 < DSA

c. Explain the mechanism underlying Digital Signature. How does it combine the strength and weakness of each encryption scheme?

- The mechanism of Digital Signature Algorithm combines the speed of message digest from SHA-1 with the scalability of public key which algorithm is much slower than hash function. The sender will hash the message to get a message digest and encrypt it with private key to digital signature. The receiver will receive both message and Digital Signature from the sender to authenticate the message.